1. Print the Boundary Nodes of a Binary Tree

```
import java.util.*;

class Main {

  static class Node {
    int data;
    Node left, right;

    Node(int data) {
      this.data = data;
    }
  }

  static void leftBoundary(Node root) {
    Node curr = root.left;

    while (curr != null) {
      if (curr.left != null || curr.right != null)
        System.out.print(curr.data + " ");

      if (curr.left != null)
        curr = curr.left;
      else
        curr = curr.right;
    }
  }

  static void leaves(Node root) {
    if (root == null)
      return;

    leaves(root.left);

    if (root.left == null && root.right == null)
      System.out.print(root.data + " ");

    leaves(root.right);
  }

  static void rightBoundary(Node root) {
    ArrayList<Integer> list = new ArrayList<>();

    Node curr = root.right;

    while (curr != null) {
      if (curr.left != null || curr.right != null)
```

```
            list.add(curr.data);

        if (curr.right != null)
            curr = curr.right;
        else
            curr = curr.left;
    }

    for (int i = list.size() - 1; i >= 0; i--)
        System.out.print(list.get(i) + " ");
  }

  static void boundary(Node root) {
    if (root == null)
        return;

    System.out.print(root.data + " ");

    leftBoundary(root);
    leaves(root);
    rightBoundary(root);
  }

  public static void main(String[] args) {

    Node root = new Node(1);

    root.left = new Node(2);
    root.right = new Node(3);

    root.left.left = new Node(4);
    root.left.right = new Node(5);

    root.right.left = new Node(6);
    root.right.right = new Node(7);

    boundary(root);
  }
}
```

2. Print the Left View of a Binary Tree

```
import java.util.*;

class Main {

  static class Node {
    int data;
    Node left, right;
```

```
    Node(int data) {
      this.data = data;
    }
  }

  static void leftView(Node root) {
    if (root == null)
      return;

    Queue<Node> q = new LinkedList<>();
    q.add(root);

    while (!q.isEmpty()) {
      int size = q.size();

      for (int i = 0; i < size; i++) {
        Node curr = q.poll();

        if (i == 0)
          System.out.print(curr.data + " ");

        if (curr.left != null)
          q.add(curr.left);

        if (curr.right != null)
          q.add(curr.right);
      }
    }
  }

  public static void main(String[] args) {

    Node root = new Node(1);

    root.left = new Node(2);
    root.right = new Node(3);

    root.left.left = new Node(4);
    root.left.right = new Node(5);

    root.right.left = new Node(6);
    root.right.right = new Node(7);

    leftView(root);
  }
}
```

3. Print the Right View of a Binary Tree

```
import java.util.*;

class Main {

  static class Node {
    int data;
    Node left, right;

    Node(int data) {
      this.data = data;
    }
  }

  static void rightView(Node root) {
    if (root == null)
      return;

    Queue<Node> q = new LinkedList<>();
    q.add(root);

    while (!q.isEmpty()) {
      int size = q.size();

      for (int i = 0; i < size; i++) {
        Node curr = q.poll();

        if (i == size - 1)
          System.out.print(curr.data + " ");

        if (curr.left != null)
          q.add(curr.left);

        if (curr.right != null)
          q.add(curr.right);
      }
    }
  }

  public static void main(String[] args) {

    Node root = new Node(1);

    root.left = new Node(2);
    root.right = new Node(3);

    root.left.left = new Node(4);
```

```
    root.left.right = new Node(5);

    root.right.left = new Node(6);
    root.right.right = new Node(7);

    rightView(root);
  }
}
```

4. Find the Lowest Common Ancestor (LCA)

```
import java.util.*;

class Main {

  static class Node {
    int data;
    Node left, right;

    Node(int data) {
      this.data = data;
    }
  }

  static Node lca(Node root, int n1, int n2) {

    if (root == null)
      return null;

    if (root.data == n1 || root.data == n2)
      return root;

    Node left = lca(root.left, n1, n2);
    Node right = lca(root.right, n1, n2);

    if (left != null && right != null)
      return root;

    if (left != null)
      return left;

    return right;
  }

  public static void main(String[] args) {

    Node root = new Node(1);

    root.left = new Node(2);
```

```
    root.right = new Node(3);

    root.left.left = new Node(4);
    root.left.right = new Node(5);

    root.right.left = new Node(6);
    root.right.right = new Node(7);

    int n1 = 4;
    int n2 = 5;

    Node result = lca(root, n1, n2);

    System.out.println(result.data);
  }
}
```

5. Find the Path from the Root to a Given Node

```
import java.util.*;

class Main {

  static class Node {
    int data;
    Node left, right;

    Node(int data) {
      this.data = data;
    }
  }

  static boolean findPath(Node root, int key,
                ArrayList<Integer> path) {

    if (root == null)
      return false;

    path.add(root.data);

    if (root.data == key)
      return true;

    if (findPath(root.left, key, path))
      return true;

    if (findPath(root.right, key, path))
      return true;
```

```
    path.remove(path.size() - 1);

    return false;
  }

  public static void main(String[] args) {

    Node root = new Node(1);

    root.left = new Node(2);
    root.right = new Node(3);

    root.left.left = new Node(4);
    root.left.right = new Node(5);

    root.right.left = new Node(6);
    root.right.right = new Node(7);

    int key = 5;

    ArrayList<Integer> path = new ArrayList<>();

    if (findPath(root, key, path))
      System.out.println(path);
    else
      System.out.println("Node not found");
  }
}
```

6. Construct a Binary Tree from Inorder and Preorder Traversal

```
import java.util.*;

class Main {

  static class Node {
    int data;
    Node left, right;

    Node(int data) {
      this.data = data;
    }
  }

  static int preIndex = 0;

  static Node buildTree(int[] inorder, int[] preorder,
                        int start, int end) {
```

```
    if (start > end)
        return null;

    Node root = new Node(preorder[preIndex++]);

    int index = start;

    while (inorder[index] != root.data)
        index++;

    root.left = buildTree(inorder, preorder,
                          start, index - 1);

    root.right = buildTree(inorder, preorder,
                           index + 1, end);

    return root;
}

static void postorder(Node root) {

    if (root == null)
        return;

    postorder(root.left);
    postorder(root.right);

    System.out.print(root.data + " ");
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();

    int[] inorder = new int[n];
    int[] preorder = new int[n];

    for (int i = 0; i < n; i++)
        inorder[i] = sc.nextInt();

    for (int i = 0; i < n; i++)
        preorder[i] = sc.nextInt();

    Node root = buildTree(inorder, preorder, 0, n - 1);
```

```
        postorder(root);
    }
}
```